



Día, Fecha:

9/04/2026

Hora de inicio:

17:20

Software Avanzado Seccion B

Kevin Raúl Pozuelos Estrada

Agenda



RECORDATORIOS



Dudas Iniciales



Anuncios Generales



Monitorization

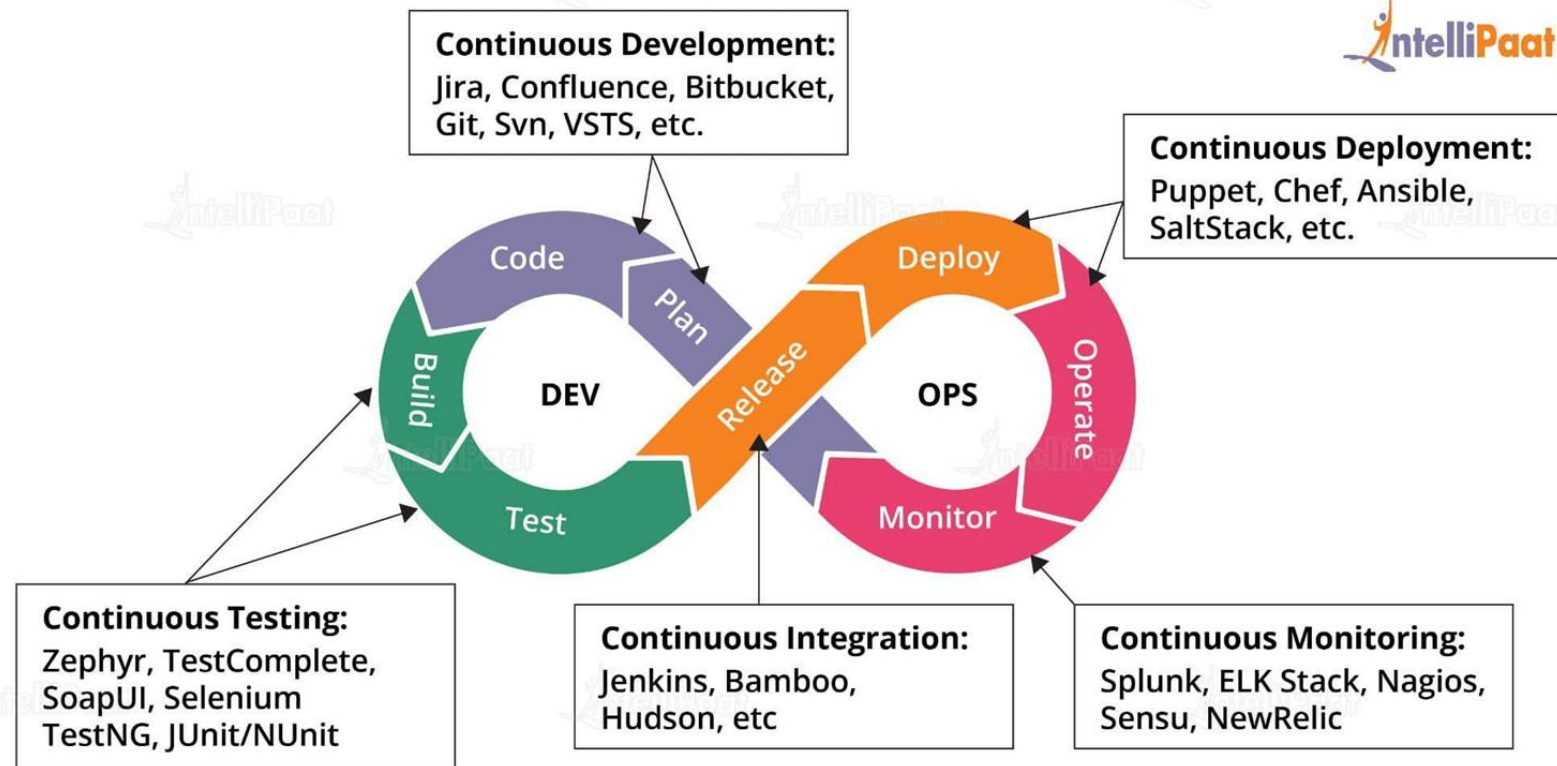


Dudas Finales



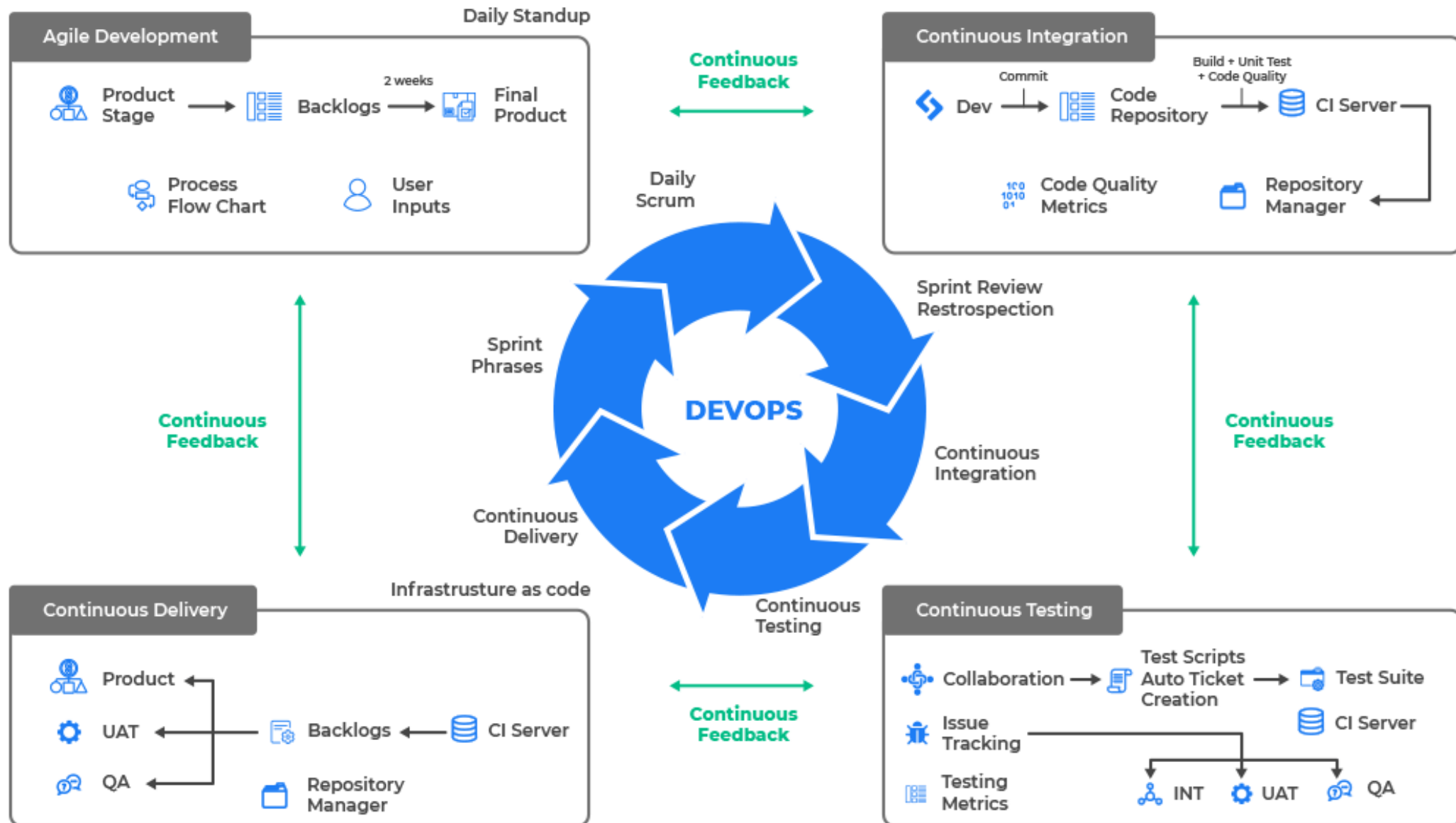
Dudas
Fase 3

Monitoring & Operate en DevOps



Monitoring en Devops

En DevOps, el monitoring (monitoreo) se refiere a la práctica de supervisar continuamente aplicaciones, servicios e infraestructuras en producción. Su objetivo principal es asegurar que los sistemas funcionen de manera óptima y estén disponibles para los usuarios finales



Características Principales



- Recolección de métricas
- Alertas y notificaciones
- Analisis de Logs
- Dashboards
- Ajustes y optimizacion

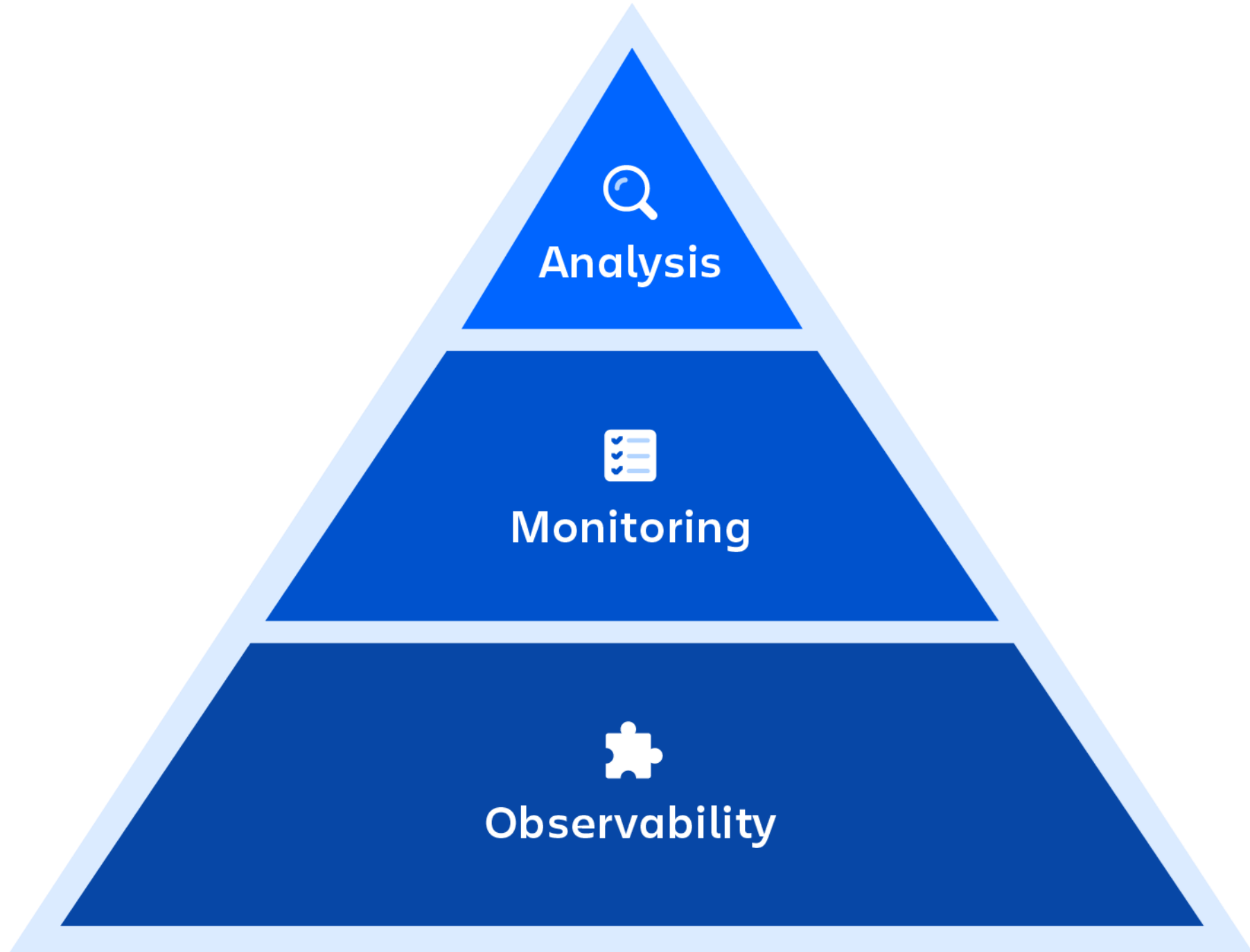
Operate en Devops



Se refiere a las actividades y prácticas relacionadas con la gestión y mantenimiento de los sistemas y aplicaciones en producción. Este aspecto es crucial para garantizar que los servicios sean estables, escalables y estén disponibles para los usuarios finales.

Características Principales

- Gestion de infraestructura
- Despliegues
- Mantenimiento y actualizaciones
- Escalabilidad
- Gestion de incidentes
- Automatizacion
- Documentacion y procedimientos

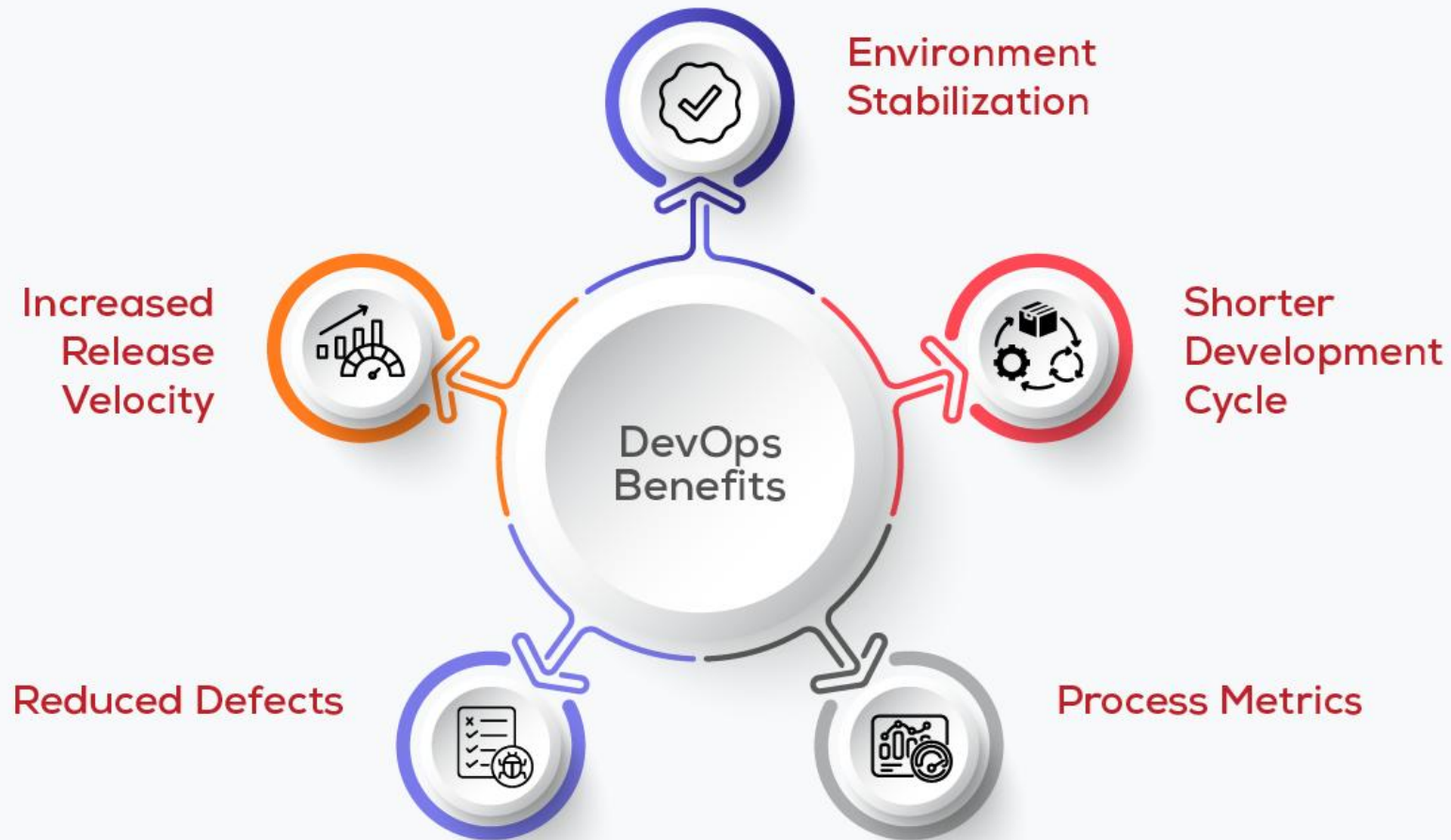




“Lo que no se
define no se puede
medir. Lo que no se
mide, no se puede
mejorar. Lo que no
se mejora, se
degrada siempre”

Frase de William Thomson Kelvin

Benefits of DevOps Monitoring in Business



Herramientas

Top DevOps Monitoring Tools



ELK Stack



elasticsearch



logstash



kibana

Logstash

Es una herramienta de ingesta de datos y procesamiento de eventos. Se utiliza principalmente para recolectar, transformar y enviar datos a diferentes destinos, como Elasticsearch

Elasticsearch

Es un motor de búsqueda y análisis basado en Lucene, diseñado para manejar grandes volúmenes de datos de manera rápida y escalable. Se utiliza comúnmente para almacenar, buscar y analizar datos en tiempo real.

Kibana

Herramienta de visualización de datos que se utiliza principalmente con Elasticsearch. Permite a los usuarios explorar, visualizar y analizar datos almacenados en Elasticsearch a través de una interfaz gráfica intuitiva

Observabilidad Unificada con OpenTelemetry y Grafana LGTM

- **El problema de la caja negra:** Análisis de por qué un sistema distribuido sin observabilidad es imposible de mantener en producción.
- **OpenTelemetry:** Integración del estándar de la industria para exportar métricas, trazas y logs desde todos los microservicios de forma unificada.
- **Stack Grafana LGTM:** Despliegue del stack completo — Loki para logs, Tempo para trazas distribuidas y Prometheus para métricas.
- **Correlación de logs con Trace ID:** Inyección de identificadores únicos en el patrón de logging para vincular eventos de consola con trazas distribuidas.
- **El puente Logback-OpenTelemetry:** Implementación del InstallOpenTelemetryAppender para redirigir el flujo de logs de consola hacia Loki.
- **Análisis visual en Grafana:** Monitoreo del sistema bajo carga real cruzando métricas, trazas y logs en un solo panel.

Monitorear VS Observar

❌ Monitoreo Tradicional

Te dice QUE algo falló.

"El servicio de inventario está caído"

"La tasa de error subió al 5%"

Pero no te dice POR QUÉ.

✅ Observabilidad

Te permite preguntar POR QUÉ falló.

"¿Qué pedido causó el error?"

"¿Qué servicio tardó más?"

Sin cambiar el código.

💡 **Clave:** En microservicios, necesitamos poder hacer preguntas que no anticipamos cuando escribimos el código. Esa es la promesa de la observabilidad.

¿Qué es OpenTelemetry?

- OpenTelemetry (o OTel) es un framework de código abierto y estándar de la industria, parte de la Cloud Native Computing Foundation (CNCF). Permite generar, recopilar y exportar datos de telemetría (métricas, trazas y logs) de aplicaciones e infraestructura para su observabilidad, de manera agnóstica a cualquier proveedor. Facilita entender el rendimiento del sistema mediante el recolector de OpenTelemetry para procesar y enviar datos a múltiples plataformas.

Características y beneficios principales:

- **Unificación:** Reúne tres señales clave de observabilidad: **trazas** (flujo de peticiones), **métricas** (datos numéricos) y **logs** (eventos), correlacionándolas para una visión integral.
- **Neutralidad:** Al ser un estándar abierto, evita la dependencia de un único proveedor de monitoreo.
- **Versatilidad:** Compatible con la mayoría de los lenguajes de programación y herramientas de observabilidad, incluyendo [Google Cloud Observability](#).
- **Instrumentación:** Permite la instrumentación automática (sin cambiar código) y manual.
- **Origen:** Surgió de la fusión de los proyectos OpenTracing y OpenCensus.

Componentes clave:

APIs y SDKs: Bibliotecas específicas para cada lenguaje para generar datos.

OpenTelemetry Collector: Un proxy que recibe, procesa y exporta los datos, adaptable como agente (sidecar) o recolector centralizado.

¿Para qué se utiliza?

- Se usa principalmente para supervisar aplicaciones nativas de la nube (como microservicios), resolver problemas de rendimiento, analizar el comportamiento del sistema distribuido y mejorar la eficiencia en el uso de recursos.

Actuador vs OpenTelemetry

Spring Actuator

-  **Pull-based:** Prometheus raspa endpoints
-  **Health checks:** /actuator/health
-  **Métricas locales:** JVM, HTTP, DB pools
-  **Ideal para:** Monitoreo de instancia

VS

OpenTelemetry

-  **Push-based:** Envía datos activamente
-  **Trazas distribuidas:** TraceID end-to-end
-  **Contexto completo:** Spans entre servicios
-  **Ideal para:** Debugging distribuido

🏆 El Veredicto

No son competidores — son complementarios.

Usaremos **Actuator** para health checks y métricas locales (Prometheus scraping),
y **OpenTelemetry** para el rastro completo de peticiones distribuidas.

Ejemplo: Cuando un pedido falla, **Actuator** te dice que el servicio está caído, y **OTel** te muestra exactamente qué petición causó el error.

Los 3 Pilares de la Observabilidad

Métricas, Trazas y Logs trabajando juntos

1 Métricas (Metrics)

Qué: Números agregados en el tiempo (latencia promedio, requests/segundo, % de CPU)

Para qué: Detectar *que* hay un problema ("La latencia subió 300%")

Ejemplo: Grafana mostrando gráficos de throughput y errores

2 Trazas (Traces)

Qué: El recorrido de UNA petición entre servicios (Trace ID + Spans)

Para qué: Entender *por qué* hay un problema ("Order Service tardó 5s esperando a Inventory")

Ejemplo: Ver el waterfall completo de un pedido lento

3 Logs (Logs)

Qué: Eventos discretos con contexto (timestamp, nivel, mensaje, TraceID)

Para qué: Detalles específicos ("Stock insuficiente para SKU-123 en Inventory Service")

Ejemplo: Logs correlacionados por TraceID para ver la historia completa

 **El poder de OTel:** Estos 3 pilares están correlacionados por TraceID. Un spike en latencia (métrica) te lleva a una traza específica, que te muestra los logs relevantes. Todo conectado.

El Flujo OTLP

- 1 **Instrumentación:** Tu app de Spring Boot genera métricas, trazas y logs
- 2 **Exportación:** Los datos se envían vía **OTLP** (gRPC o HTTP)
- 3 **Collector (Opcional):** Un intermediario que procesa, filtra y enruta datos
- 4 **Backend:** Grafana Tempo, Jaeger, o cualquier backend compatible los almacena
- 5 **Visualización:** Grafana te muestra gráficos, traces, y logs unidos

💡 **Simplicidad:** En desarrollo, podemos saltar el Collector y enviar directamente a Tempo o Jaeger. En producción, el Collector nos da control fino sobre qué enviar y adónde.

OpenTelemetry: Resumen

-  **Observabilidad > Monitoreo:** Permite preguntar "por qué" sin código nuevo
-  **OTel = Estándar Universal:** Un protocolo (OTLP) para todos los backends
-  **Spring Boot 4:** Observabilidad nativa sin agentes externos
-  **3 Pilares:** Métricas (qué), Trazas (por qué), Logs (detalles)
-  **Actuator + OTel:** Complementarios, no competidores

Logging



El logging es el registro estructurado de eventos que ocurren dentro del sistema.



No solo errores. Incluye:



Eventos de negocio (usuario compró)



Eventos técnicos (request HTTP)



Estados internos (conexión DB, cache, etc.)

Tipos de logs

Logs técnicos

Errores (ERROR)

Advertencias (WARN)

Información (INFO)

Debug (DEBUG)

◆ Logs de negocio

“Usuario creó cuenta”

“Pago rechazado”

“Pedido completado”

Logging estructurado

Formato JSON

Campos consistentes

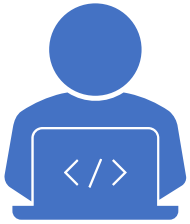
Fácil de indexar (ej: Elasticsearch)

Beneficios:

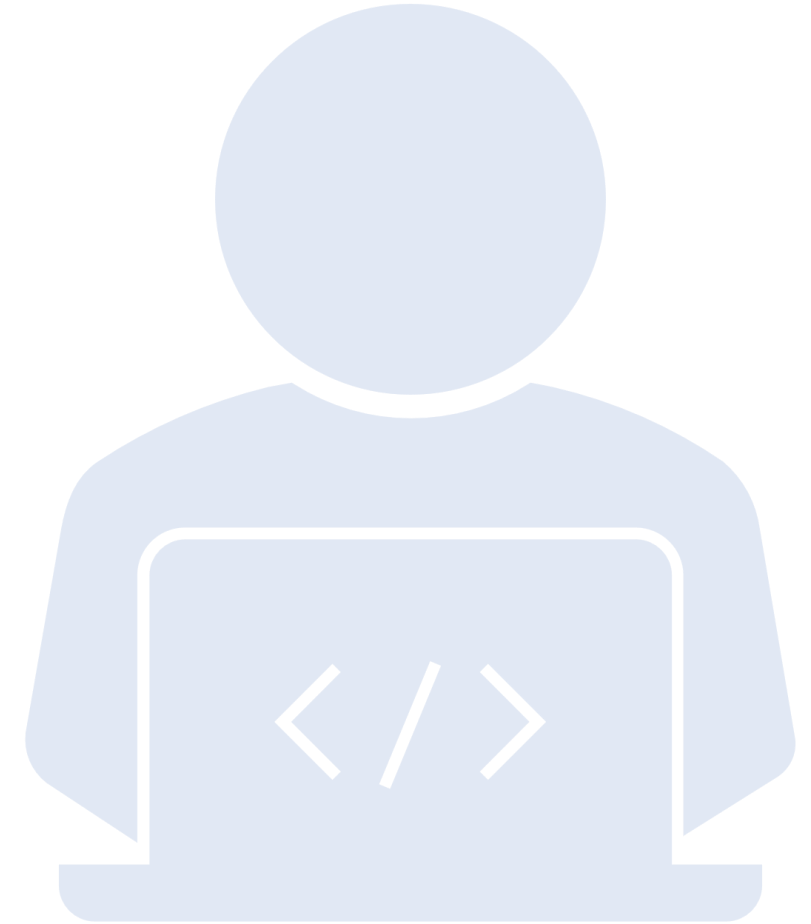
Filtrar rápido

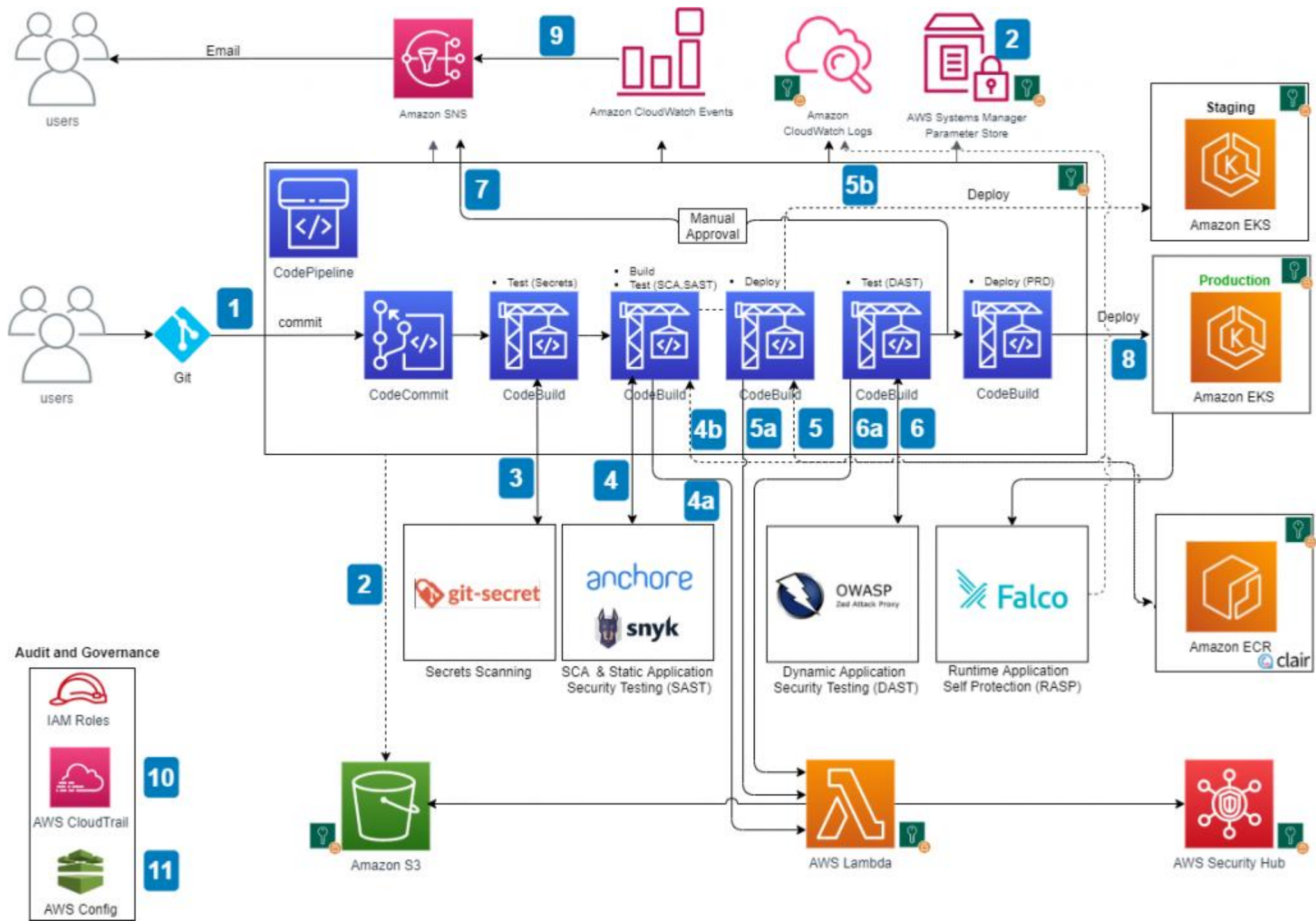
Hacer dashboards

Correlacionar eventos



Los logs no son
para escribir
código... son para
entender sistemas.







**¡GRACIAS POR SU
ATENCIÓN!**

RECUERDA QUE TENEMOS NUESTRO FORO SEMANAL DONDE PUEDES CONSULTAR CUALQUIER DUDA
QUE TE SURJA EN LA SEMANA